جامعة ابن طفيل
Université Ibn Tofail

ANNÉE ACADÉMIQUE

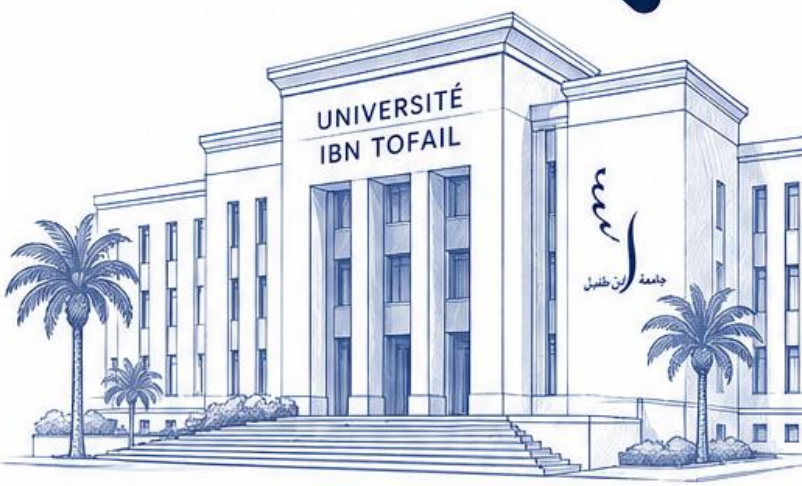
2025 - 2026

PROJET

ANALYSE DES SENTIMENTS DANS LES AVIS

Réalisée Par : Oumaima Bendjaj





1.Introduction :

Ce document présente un rapport détaillé du projet d'analyse des sentiments, réalisé dans le cadre de l'apprentissage des notions fondamentales du traitement automatique du langage naturel (NLP). L'objectif principal de ce projet était de concevoir un système capable de classifier les avis des utilisateurs en deux catégories, « positif » ou « négatif », à partir de textes préalablement prétraités.

2.Objectifs :

1. Prétraiter et analyser des données textuelles (avis clients).

2. Représenter les textes sous forme numérique à l'aide de la méthode TF-IDF.

3. Utiliser un algorithme de classification supervisée pour prédire si un avis est positif ou négatif.

4. Évaluer la performance du modèle avec des métriques comme la précision et le rappel.

3.Déroulement et Méthodologie :

1. **Chargement et exploration des données**
   Le jeu de données utilisé provient de l'ensemble **IMDb Reviews**, qui contient des avis de films classés en deux catégories : *positif* et *négatif*.
   Une première analyse des données a été réalisée à l'aide des commandes `head()` et `info()` afin de comprendre la structure du dataset, vérifier la répartition des avis et identifier les colonnes importantes, notamment **review** et **sentiment**.

2. **Prétraitement des données**
   Plusieurs étapes de nettoyage ont été appliquées aux textes afin d'améliorer la qualité des données :

   - Conversion des textes en minuscules pour uniformiser les mots ;
   - Suppression des caractères spéciaux et des éléments inutiles afin de réduire le bruit ;
   - Élimination des mots vides (*stopwords*) grâce à la bibliothèque NLTK, en retirant des mots fréquents mais peu significatifs comme *"the"* ou *"and"*.

3. **Division des données**
   Le jeu de données a été séparé en deux parties :

   - **80 %** des données pour l'entraînement du modèle ;
   - **20 %** des données pour tester et évaluer les performances du modèle.

4. **Transformation TF-IDF** :
   La méthode **TF-IDF (Term Frequency – Inverse Document Frequency)** a été utilisée pour transformer les avis textuels en vecteurs numériques.

   Cette technique permet de représenter chaque mot selon son importance dans un avis par rapport à l'ensemble des documents.

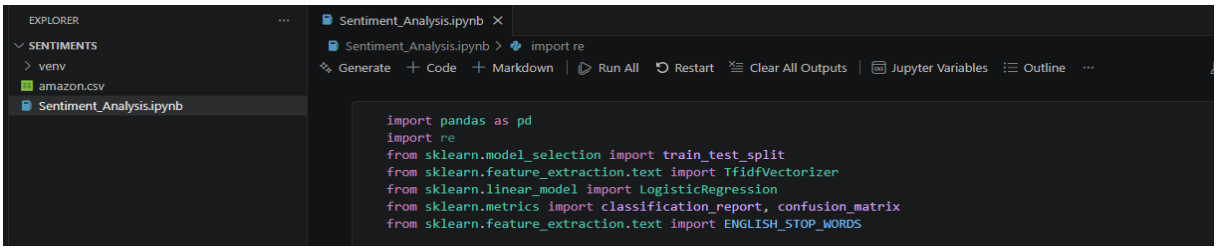
7.Technologie utilisée :

L'interface utilisateur a été développée avec **Gradio**, une bibliothèque Python qui facilite la création rapide d'interfaces web interactives et conviviales.

Fonctionnalités principales :

- Permettre à l'utilisateur de saisir un avis textuel ;
- Afficher instantanément la prédiction du modèle (*positif* ou *négatif*) ;
- Offrir une interface simple, intuitive et accessible directement depuis un navigateur web.

4.Code Python :




```python
# 1. Charger le dataset

def charger_dataset():
    df = pd.read_csv("amazon.csv")
    print(df.head())
    print(df.info())
    return df


# Stopwords

stop_words = set(ENGLISH_STOP_WORDS)

def supprimer_stopwords(texte):
    mots = texte.split()
    mots_filtrés = [mot for mot in mots if mot not in stop_words]
    return " ".join(mots_filtrés)
```

```python
# 2. Nettoyer les données

def nettoyer_donnees(df):
    df["review"] = df["review"].str.lower()
    df["review"] = df["review"].str.replace(r"[^a-zA-Z\s]", "", regex=True)
    df["review"] = df["review"].apply(supprimer_stopwords)
    return df
```

```python
# 3. Séparer les données

def separer_donnees(df):
    X_train, X_test, y_train, y_test = train_test_split(
        df["review"], df["resultat"], test_size=0.2, random_state=42
    )
    return X_train, X_test, y_train, y_test
```

```python
# 4. Vectorisation TF-IDF

def vectoriser_texte(X_train, X_test):
    vectoriseur = TfidfVectorizer()

    X_train_tfidf = vectoriseur.fit_transform(X_train)
    X_test_tfidf = vectoriseur.transform(X_test)

    return X_train_tfidf, X_test_tfidf, vectoriseur
```

```python
# 5. Entraîner le modèle

def entrainer_modele(X_train_tfidf, y_train):
    modele = LogisticRegression()
    modele.fit(X_train_tfidf, y_train)
    return modele
```

```python
# 6. Évaluer le modèle

def evaluer_modele(modele, X_test_tfidf, y_test):
    y_pred = modele.predict(X_test_tfidf)

    print("\nRapport de classification:\n")
    print(classification_report(y_test, y_pred))

    print("\nMatrice de confusion:\n")
    print(confusion_matrix(y_test, y_pred))
```

Description :

Gradio :

- Interface simple et conviviale facilitant l'utilisation par tous les utilisateurs.
- Zone de saisie permettant à l'utilisateur d'entrer un avis ou un commentaire.
- Bouton de validation pour lancer l'analyse du texte saisi.
- Affichage instantané du résultat de prédiction (« positif » ou « négatif »).
- Interface accessible directement via un navigateur web grâce à Gradio.
- Design interactif offrant une expérience utilisateur fluide et rapide.

```python
import re
import gradio as gr


# 7. Prédire le sentiment

def predire_sentiment(avis, modele, vectoriseur):
    # Nettoyage
    avis_nettoye = avis.lower()
    avis_nettoye = re.sub(r"[^a-zA-Z\s]", "", avis_nettoye)
    avis_nettoye = supprimer_stopwords(avis_nettoye)

    # Vectorisation
    avis_tfidf = vectoriseur.transform([avis_nettoye])

    # Prédiction
    prediction = modele.predict(avis_tfidf)[0]

    return f"Résultat : {prediction}"
```

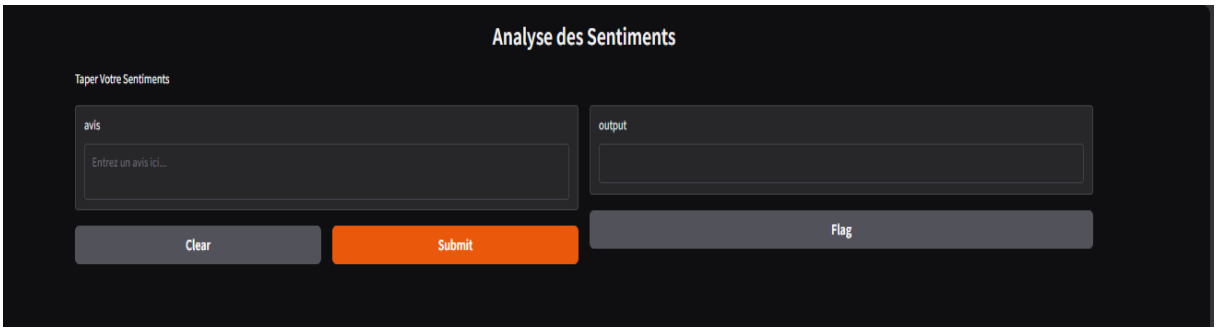
```python
# 8. Interface Gradio
def lancer_application():
    # Charger les données
    df = charger_dataset()
    df = nettoyer_donnees(df)
    # Split
    X_train, X_test, y_train, y_test = separer_donnees(df)
    # TF-IDF
    X_train_tfidf, X_test_tfidf, vectoriseur = vectoriser_texte(X_train, X_test)
    # Modèle
    modele = entrainer_modele(X_train_tfidf, y_train)
    # Évaluation
    evaluer_modele(modele, X_test_tfidf, y_test)

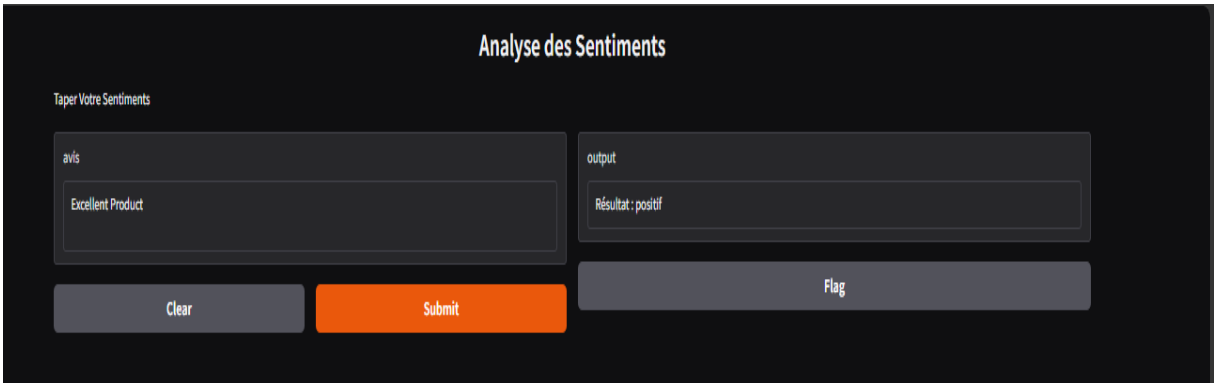

    # Fonction pour interface
    def interface_gradio(avis):
        return predire_sentiment(avis, modele, vectoriseur)

    # Interface Gradio
    app = gr.Interface(
        fn=interface_gradio,
        inputs=gr.Textbox(lines=2, placeholder="Entrez un avis ici..."),
        outputs="text",
        title="Analyse des Sentiments",
        description="Taper Votre Sentiments"
    )
    app.launch()
```

5.Interface graphique :

L'interface développée avec Gradio est simple, rapide et facile à utiliser. Elle permet à l'utilisateur de saisir un avis et d'obtenir instantanément une prédiction, offrant ainsi une expérience utilisateur fluide et interactive.





Analyse des résultats :

- **Le modèle de Régression Logistique** a été entraîné sur des avis clients transformés avec la méthode TF-IDF.
- Les résultats obtenus montrent que le modèle est capable de distinguer correctement les avis positifs et négatifs.
- **Accuracy :** L'accuracy mesure le pourcentage de bonnes prédictions. Une valeur élevée indique une bonne performance globale.
- **Precision :**La précision indique combien de prédictions positives sont réellement positives.
- **Recall :**Le rappel mesure la capacité du modèle à retrouver les vrais avis positifs.
- **F1-score :**Le score F1 représente un compromis entre la précision et le rappel.
- **La matrice de confusion** permet également de visualiser les erreurs de classification réalisées par le modèle.

Résultat :

```
Data columns (total 3 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   review     19996 non-null  object
 1   sentiment  19996 non-null  int64
 2   resultat   19996 non-null  object
dtypes: int64(1), object(2)
memory usage: 468.8+ KB
None

Rapport de classification:

              precision    recall  f1-score   support

     négatif       0.86      0.62      0.72       978
     positif       0.89      0.97      0.93      3022
...
Matrice de confusion:

[[ 610  368]
 [  99 2923]]
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

6.Conculusion et Amélioration :

Ce projet a permis d’appliquer les concepts fondamentaux du Machine Learning et du traitement automatique du langage.

L’utilisation de TF-IDF et de la Régression Logistique a donné des résultats satisfaisants pour la classification des sentiments.

Une amélioration possible serait :

- d’utiliser un dataset plus grand,

- d’ajouter une classe neutre,

- ou de tester d’autres algorithmes comme SVM ou Naive Bayes.